Customer Churn Prediction

A Machine Learning Study on the Cell2Cell Telecom Dataset

Academic Project Report (Projektarbeit)

Supervisor: Prof. Dr. Nadia Bauer

Name : Ali Elshafey
Fachhochschule Dortmund
Date of submission: June 2026

Table of Contents

1. Introduction *1*

1.1 Background and Motivation **1**

1.2 Research Questions **1**

1.3 Structure of the Report **2**

2. State of Research *3*

3. Theoretical Background *5*

3.1 Binary Classification **5**

3.1.1 Logistic Regression 5

3.1.2 Random Forest 6

3.1.3 XGBoost 7

3.1.4 LightGBM 8

3.2 Strategies for Imbalanced Data **9**

3.3 SMOTE **9**

3.4 Evaluation Metrics **10**

3.5 Cross Validation **11**

3.6 Hyperparameter Tuning **12**

3.7 Model Interpretation and Feature Importance **13**

4. Methodology *14*

5. Dataset and Data Preparation *15*

5.1 Dataset Description **15**

5.2 Data Understanding and Cleaning **17**

5.3 Missing Value Treatment **18**

5.4 Feature Engineering and Imputation **19**

6. Exploratory Data Analysis and Feature Selection *20*

7. Modelling and Results *24*

7.1 Baseline Model Comparison **24**

ROC Models curves **24**

7.2 Threshold Optimisation **25**

7.3 SMOTE Evaluation **26**

7.4 Hyperparameter Tuning **27**

7.5 Model Interpretation **28**

7.5.1 Built-in Feature Importance 28

7.5.2 Permutation Importance 29

7.6 Final Model Performance **30**

8. Discussion *32*
8.1 Interpretation of Findings **32**
8.2 Answers to Research Questions **32**
8.3 Practical Implications **33**
9. Limitations *34*
10. Conclusion *35*
11. References *36*

1. Introduction

1.1 Background and Motivation

Mobile phone companies make most of their money from customers who stay with them for a long period of time. The cost of attracting a new customer through advertising, sign-up offers and free devices is much higher than the cost of keeping an existing one. For this reason, the moment a customer cancels a contract, the company loses not only this month's bill but also all the future months that customer would have paid for. This kind of customer loss is known in the industry as churn (Verbeke et al., 2012).

In a saturated market like the telecom sector, where most adults already own a mobile contract, growth no longer comes mainly from new sign-ups. It comes from keeping the customers you already have. A company that can identify which customers are likely to leave in the near future has the chance to act before it is too late, for example by offering a better tariff, a phone upgrade or a personal call from a retention agent. Without such a system, the company can only react after the cancellation, which is normally too late.

Further information about the Cell2Cell dataset and its source can be found in the references section.

1.2 Research Questions

The aim of this project is to build a machine learning model that estimates, for each customer in the Cell2Cell dataset, the probability that they will cancel their contract. From a technical point of view this is a supervised binary classification problem: every customer is labelled either as a churner (the customer left) or as a non-churner (the customer stayed), and the model has to learn the patterns that separate these two groups. From a business point of view, the goal is to give the retention team a ranked list of customers so that limited retention budget can be spent on the people who are really at risk of leaving.

On this basis, the project is guided by the following central research question:

To what extent can a supervised machine learning model, trained only on the behavioural, contractual and demographic attributes available in the Cell2Cell dataset, predict customer churn with sufficient quality to support a real-world retention strategy?

Three sub-questions structure the empirical work that answers it:

- RQ1: Which family of classifiers, linear models, bagging-based ensembles or gradient boosting, produces the best ranking of customers by churn risk on this dataset, and how large is the gap between them?
- RQ2: Given the moderate class imbalance of roughly 29% churners, which combination of class weighting, decision-threshold tuning and synthetic oversampling yields the most useful trade-off between recall and precision from a business perspective?
- RQ3: Which customer attributes drive the predictions of the final model, and are these drivers consistent across complementary interpretation methods such as built-in feature importance and permutation importance?

1.3 Structure of the Report

The remainder of this report is organised so that the general theory is kept separate from the project-specific work. Section 2 reviews the existing research on churn prediction in the telecommunications sector. Section 3 presents the theoretical background, the classification problem, the four model families, the treatment of class imbalance, the evaluation metrics, the validation strategy, the tuning procedure and the interpretation methods, independently of the dataset. Section 4 summarises the methodological workflow and the software used. Section 5 describes the dataset, its cleaning, the treatment of missing values and the preprocessing. Section 6 presents the exploratory data analysis and the feature selection. Section 7 forms the core of the practical part, covering the baseline model comparison, the threshold and SMOTE experiments, the hyperparameter tuning including an ensemble evaluation, the model interpretation and the final model performance. Section 8 discusses the findings and answers the research questions, Section 9 sets out the limitations, and Section 10 draws the overall conclusion. RQ1 is answered in Sections 7.1 and 8, RQ2 in Sections 7.2 and 7.3, and RQ3 in Section 7.5.

2. State of Research

Customer churn prediction is one of the oldest and most studied applications of supervised machine learning in the business analytics literature, and the telecommunications sector has been at the centre of this research for more than two decades. The reason for this prominence is economic: as Verbeke et al. (2012) point out, the cost of acquiring a new mobile subscriber in a saturated market is several times higher than the cost of retaining an existing one, so even a small improvement in retention has a large effect on profitability. Early academic work on the problem therefore concentrated on two questions: which statistical or learning method predicts churn most accurately, and which customer attributes drive the prediction.

The benchmark study by Neslin et al. (2006), based on the same Cell2Cell data used in this project, is generally regarded as the starting point of the modern churn-prediction literature. The authors organised a prediction competition in which academic and industry teams applied a wide range of techniques to a common dataset and measured their lift on the top deciles of predicted risk. Their two central findingshave remained influential since. First, the choice of modelling technique matters less than is often assumed: well-tuned logistic regression, decision trees and neural networks produced broadly comparable results on the same data. Second, the practical value of a churn model lies in its ability to order customers by their predicted churn probability, not in the absolute accuracy of its predictions, because retention budgets are limited and only the highest-risk customers can realistically be contacted.

A second strand of the literature focuses on the imbalanced nature of churn data. In most telecom datasets, including Cell2Cell, the share of churners lies between 2% and 30%, which means that accuracy alone is a misleading metric and that standard classifiers tend to be biased towards the majority class. Chawla et al. (2002) introduced SMOTE, the Synthetic Minority Over-sampling Technique, which generates synthetic minority examples by interpolating between existing ones. SMOTE became one of the most widely used remedies for class imbalance, but subsequent empirical work by López et al. (2013) has shown that its benefit is far from guaranteed: when the imbalance is only moderate and the underlying classifier already accepts class weights, oversampling often produces no improvement or even degrades the ranking quality. The same paper recommends combining several strategies,namely class weighting, threshold tuning and, where appropriate, resampling rather than relying on a single technique.

A third and more recent strand has shifted the methodological focus from linear or single-tree models to gradient boosting. The XGBoost system (Chen and Guestrin, 2016) and the LightGBM library (Ke et al., 2017) both implement scalable variants of gradient-boosted decision trees and have repeatedly been reported as state of the art for classification on tabular data, including churn. Their advantage on this kind of data is that they combine many weak, individually uninformative features into a strong joint prediction, which fits the empirical observation that churn is multi-factorial: in such datasets individual correlations with the target are typically weak, but the combined signal across dozens of features is substantially stronger.

Alongside accuracy, the recent literature has paid increasing attention to interpretability. A churn score is only acted upon by a retention team if the team understands which customer characteristics drove the prediction. Permutation importance, introduced in the random-forest context by Breiman

(2001) and now available as a model-agnostic procedure in scikit-learn (Pedregosa et al., 2011), measures the drop in validation performance when each feature is randomly shuffled in turn. SHAP values, proposed by Lundberg and Lee (2017), provide a more granular, per-prediction decomposition based on cooperative game theory. The general consensus in applied work is that built-in importance scores, which simply count how often a feature is used inside the trees, are useful as a first overview but should be complemented by at least one model-agnostic method, because tree-based importance can be biased towards high-cardinality variables.

The literature converges on three points. The most predictive variables in telecom churn studies are consistently tenure, equipment age, customer-service interactions and recent changes in usage, with billing-related variables playing a secondary role. Reported validation ROC-AUC values on Cell2Cell and on comparable public datasets typically lie in the range of 0.60 to 0.75, depending on preprocessing choices and on whether external information is available. Finally, the gain from sophisticated hyperparameter tuning is usually modest compared to the gain from choosing an appropriate model family and from carefully handling class imbalance and the decision threshold. The present project is positioned within this body of work: it applies the now-standard combination of gradient boosting, class weighting and threshold tuning to the Cell2Cell dataset, and it uses both tree-internal feature importance and permutation importance to interpret the resulting model.

3. Theoretical Background

This section presents the concepts and methods used in the project independently of the Cell2Cell data. It defines the classification task, introduces the four model families, explains the treatment of class imbalance, derives the evaluation metrics, sets out the validation strategy, describes the tuning procedure and explains the two interpretation methods. The project-specific application of each concept is given in the practical sections (5 to 7) and is referenced back to here rather than repeated.

3.1 Binary Classification

In supervised learning a model is trained on examples for which both the input features and the correct output label are known, and the goal is to learn a mapping that generalises to unseen examples. Churn prediction is a supervised binary classification problem: each customer is described by a feature vector and is labelled either as a churner or as a non-churner, and the model has to estimate, for a new customer, the probability of belonging to the churn class. Because retention budgets in practice are limited, the quality of the probability ranking, ordering customers from most to least likely to churn, is at least as important as the quality of the final binary decision. This priority shapes the choice of the primary evaluation metric in Section 3.4 and the choice of the decision threshold in Section 7.2.

Four model families were selected for comparison: one linear model (Logistic Regression), one bagging-based ensemble (Random Forest) and two gradient-boosting systems (XGBoost and LightGBM). This combination covers the model classes that the churn-prediction literature has consistently shown to be competitive, and it allows the additional value of more complex methods to be measured against simpler baselines.

3.1.1 Logistic Regression

Logistic regression is a linear classifier that models the probability of churn directly. For a customer described by a feature vector x, the model first forms a linear combination of the features,

$$z = w^{\mathrm{T}}x + b$$

where w is the learned weight vector and b the bias term. This score is then mapped to a probability through the logistic sigmoid function:

$$\sigma(z) = 1 / (1 + e^{-z})$$

so that the predicted churn probability is

$$P(y = 1 \mid x) = \sigma(w^{\mathrm{T}}x + b)$$

The sigmoid squeezes any real-valued score into the interval (0, 1), which is what allows the linear score to be interpreted as a probability. A large positive weight on a feature such as CurrentEquipmentDays means that an older device pushes the predicted churn probability upward, while a negative weight on MonthsInService means that longer tenure pushes it down. The sign and magnitude of each coefficient are therefore directly interpretable, which is the main reason logistic regression is used here as a transparent reference point.

The parameters w and b are estimated by maximising the likelihood of the observed labels, which is equivalent to minimising the binary cross-entropy (log-loss) over the n training customers:

$$L(w) = -(1/n)\,\Sigma_i\,[\,y_i\,log\,p_i + (1 - y_i)\,log(1 - p_i)\,]$$

Where

$$p_i = \sigma(w^T x_i + b)$$

An $L2$ penalty $\lambda\|w\|^2$ is usually added to this objective to shrink the coefficients and reduce overfitting; the regularisation strength is the main hyperparameter, controlled in scikit-learn through the inverse parameter C (Hastie, Tibshirani and Friedman, 2009, ch. 4). Because the decision boundary is linear in the input space, logistic regression cannot represent interactions between features unless these are engineered manually, a real limitation in churn data, where the predictive signal is spread across many weakly interacting variables.

3.1.2 Random Forest

Both Random Forest and the gradient-boosting systems described below are built from decision trees, so the criterion by which a tree chooses its splits is introduced here first. A decision tree partitions the data recursively: at each node it searches for the split that best separates churners from non-churners. The quality of a candidate split is measured by the reduction in impurity it produces, a quantity known as the information gain (Breiman et al., 1984).

The impurity of a node measures how mixed the two classes are within it. For a binary churn problem, let p be the proportion of churners in a node. The two standard impurity measures are the Gini impurity,

$$G = 1 - (p^2 + (1 - p)^2) = 2p(1 - p),$$

used by the CART algorithm (Breiman et al., 1984), and the entropy,

$$H = -p\,log_2\,p - (1 - p)\,log_2(1 - p),$$

which stems from information theory (Shannon, 1948). Both reach their maximum when the node is evenly split between churners and non-churners (maximum uncertainty) and their minimum of zero when the node contains only one class (a pure node).

The information gain of a split is then the impurity of the parent node minus the weighted average impurity of the two child nodes it creates:

$$IG = I(parent) - (\,(n_{left}/\,n) \cdot I(left) + (n_{right}\,/\,n) \cdot I(right)\,)$$

where I is the chosen impurity measure, n is the number of customers in the parent node, and n_{Lift} and n_{right} are the numbers falling into the left and right child nodes (Breiman et al., 1984). A split that produces purer children, groups that are more clearly churners or more clearly non-churners, has a larger information gain, and the tree greedily selects the split with the highest gain at each

node. In the churn context, a split such as *CurrentEquipmentDays > 400 days* is chosen precisely because separating customers with older devices yields child nodes whose churn rates differ markedly from the parent, lowering the overall impurity.

This gain criterion is central to the rest of the project for two reasons. First, it is the same quantity that the gradient-boosting models in Sections 3.1.3 and 3.1.4 maximise when growing their trees. Second, it is the basis of the built-in feature-importance measure discussed in Section 3.7, where the importance of a feature is the total information gain of all splits that use it. A known limitation of the gain criterion is that it is biased towards features with many distinct values, because such features offer more candidate split points and can accumulate gain even without genuine predictive relevance (Breiman et al., 1984); this bias is the reason the project complements built-in importance with permutation importance in Section 3.7.

Random Forest is a bagging ensemble of decision trees designed to reduce the high variance of a single tree (Breiman, 2001). It introduces randomness in two independent ways. First, each of the B trees is trained on a bootstrap sample, a dataset of the same size as the original, drawn with replacement, so that each tree sees a slightly different set of customers. Second, at every split node only a randomly chosen subset of the features (typically $\sqrt{m}$ out of m features) is considered as candidate split variables. This second step is what distinguishes Random Forest from plain bagging: it prevents a few dominant features from being chosen in every tree, which would make the trees highly correlated and defeat the purpose of averaging. The final prediction is the average of the individual tree predictions:

$$\hat{P}(y = 1 \mid x) = (\frac{1}{B}) \Sigma^{b} T^{b}(x)$$

where $T^{b}(x)$ is the churn probability returned by the b-th tree. Because the trees are deliberately decorrelated, averaging their outputs reduces the variance of the ensemble without increasing its bias, which is why a forest generalises far better than any single deep tree (Breiman, 2001). The main hyperparameters are the number of trees B, the maximum depth of each tree, the minimum number of samples required to split a node, and the number of features sampled per split. Random Forest is robust to outliers and to the skewed, heavy-tailed distributions typical of telecom usage data, and it rarely overfits even with default settings, which makes it a strong non-linear baseline.

The same averaging principle extends beyond a single algorithm: a voting ensemble combines the predicted probabilities of several different classifiers, on the assumption that models with different inductive biases make partially uncorrelated errors, so that averaging their outputs can reduce variance further. This idea is applied in Section 7.4, where a voting ensemble of the three tree-based models is evaluated.

3.1.3 XGBoost

XGBoost is a gradient-boosted tree system in which trees are added sequentially, each one correcting the errors of the current ensemble, rather than independently as in Random Forest (Chen

and Guestrin, 2016). After $t - 1$ trees the prediction for customer i is $\hat{y}_i^{(t-1)}$, and the new tree f_t is chosen to minimise the regularised objective:

$$L^{(t)} = \Sigma_i \, l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t), \;\; \Omega(f) = \gamma T + \tfrac{1}{2} \lambda \|w\|^2$$

Here l is a differentiable loss function (log-loss for churn classification), T is the number of leaves in the new tree and w are the leaf weights. The regularisation term Ω is the key contribution: γ penalises each additional leaf, controlling how complex the tree may become, and λ penalises large leaf weights through an L2 term, smoothing the predictions and preventing overfitting (Chen and Guestrin, 2016). When γ and λ are set to zero the objective reduces to ordinary gradient boosting.

To choose splits efficiently, XGBoost uses a second-order Taylor expansion of the loss, computing for each customer the first-order gradient g_i and second-order gradient h_i. The optimal weight of a leaf and the gain of a candidate split follow in closed form from the sums of these gradients over the customers falling into each leaf, which is what makes the system both fast and theoretically grounded. Two further mechanisms control overfitting: shrinkage, which multiplies each new tree's contribution by a learning rate η so that no single tree dominates, and column subsampling, borrowed from Random Forest. A practical advantage is that XGBoost handles missing values natively by learning a default split direction for absent entries, so it does not strictly require imputation. The most influential hyperparameters are the learning rate η, the maximum tree depth, the number of boosting rounds, the regularisation terms λ and γ, and the row/column subsampling ratios; their tuning is described in Section 3.6 and the result in Section 7.4 (Chen and Guestrin, 2016).

3.1.4 LightGBM

LightGBM is also a gradient-boosted decision tree system, built on the same additive boosting principle as XGBoost, but engineered for far greater speed on large, high-dimensional data (Ke et al., 2017). It differs from standard gradient boosting in four ways.

Histogram-based split finding. Instead of evaluating every distinct feature value as a potential split, LightGBM buckets each continuous feature into a small number of discrete bins and builds a histogram of gradient statistics per bin. Finding the best split then requires scanning only the bins, reducing the cost from being proportional to the number of distinct values to being proportional to the much smaller number of bins (Ke et al., 2017).

Leaf-wise growth. Where standard implementations grow trees level by level, LightGBM grows leaf-wise: at each step it splits the single leaf that promises the largest loss reduction. This produces more accurate trees for the same number of leaves, but the resulting trees can become deep and unbalanced, so they overfit on small datasets unless the number of leaves and the minimum samples per leaf are constrained.

Gradient-based One-Side Sampling (GOSS). LightGBM uses the gradient magnitude as a proxy for how poorly an instance is currently predicted. GOSS keeps all instances with large gradients and randomly samples only a fraction of those with small gradients, then re-weights the sampled small-gradient instances so that the data distribution is not distorted. Because large-gradient instances contribute most to the information gain, this yields a more accurate gain estimate than uniform random sampling at the same sampling rate (Ke et al., 2017).

Exclusive Feature Bundling (EFB). In sparse feature spaces many features rarely take nonzero values simultaneously. EFB bundles such mutually exclusive features into a single feature by assigning them disjoint value ranges, reducing the effective number of features. Finding the optimal bundling is NP-hard, so LightGBM reduces it to a graph-colouring problem solved with a greedy algorithm (Ke et al., 2017).

Across five public datasets the authors report that LightGBM accelerated training by up to roughly twenty times over the baseline GBDT while achieving almost identical AUC (Ke et al., 2017). For this project the faster training is the main practical reason LightGBM was chosen as the deployed model, since it makes periodic retraining on fresh data inexpensive. Its key hyperparameters mirror those of XGBoost, learning rate, number of leaves, maximum depth, minimum child samples and the $L1/L2$ regularisation terms.

3.2 Strategies for Imbalanced Data

Class imbalance means that one class is much more common than the other. When the minority class is rare, a classifier trained with the default objective tends to favour the majority class, and accuracy becomes a misleading measure of quality (see Section 3.4). Two main strategies are used to counter this. The first is to give the rare class a higher weight during training, so that each minority example contributes more to the loss; in scikit-learn this is the class_weight = 'balanced' option, which weights each class inversely to its frequency. The second is to rebalance the training set by generating synthetic minority examples, which is the subject of Section 3.3. Both strategies are applied and compared on the Cell2Cell data in Sections 7.1 to 7.3 (Pedregosa et al., 2011).

3.3 SMOTE

The most widely used resampling method is SMOTE, the Synthetic Minority Over-sampling Technique (Chawla et al., 2002), which over-samples the minority class by generating synthetic examples instead of duplicating existing ones.Duplication simply copies minority observations, which lets a classifier memorise the repeated points and tends to produce overfitting on those exact rows; SMOTE instead creates new, previously unseen minority points and thereby enlarges the region of feature space the model associates with the minority class.

The construction is geometric. For a minority observation x_i, SMOTE finds its k nearest neighbours among the other minority points and selects one of them at random, x_j. A synthetic example is then placed at a random position on the straight line connecting the two:

$$x_{new} = x_{\mathrm{i}} + \delta \cdot (x_j - x_{\mathrm{i}}), \delta \in [0, 1]$$

with δ drawn uniformly from [0, 1],

Because δ lies between 0 and 1, the new point falls somewhere on the segment between the original observation and its neighbour, so the synthetic churners are interpolations of real churners rather than copies. Repeating this for many minority points until the classes are balanced is what produced, in this project, the increase of the training set from 40,837 to 58,136 rows reported in Section 7.3.

The same mechanism explains the method's known weakness. The interpolation is performed using only minority points and is unaware of where the majority class lies, so synthetic examples generated near the decision boundary can be placed inside majority-class territory and blur the separation between the two classes. The original authors evaluate the technique in ROC space using the area under the curve (Chawla et al., 2002), and later empirical work shows that the benefit is not guaranteed: on only moderately imbalanced data, and especially when the classifier already supports class weighting, SMOTE often fails to improve and can even degrade the ranking quality (López et al., 2013). For this reason SMOTE is treated in this project as one candidate strategy to be tested rather than assumed to help; its application to the Cell2Cell data and the comparison against class weighting are reported in Section 7.3.

3.4 Evaluation Metrics

All five metrics used in this project are derived from the confusion matrix, which cross-tabulates the model's predictions against the true churn labels. For a binary churn problem the matrix has four cells: true positives (TP), churners correctly flagged as churners; true negatives (TN), stayers correctly identified as stayers; false positives (FP), stayers incorrectly flagged as churners; and false negatives (FN), churners the model missed. The five reported metrics are different summaries of these four numbers (Manning, Raghavan and Schütze, 2008, ch. 8). The confusion matrix is shown in Table 1.

	Predicted churn	**Predicted stay**
Actual churn	TP	FN
Actual stay	FP	TN

Table 1: Confusion matrix for the binary churn problem.

Accuracy is the share of all predictions that are correct:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

Although intuitive, accuracy is misleading when the classes are imbalanced, because a trivial model that predicts the majority class for everyone can score highly while identifying not a single churner. A metric that a useless model can score highly on cannot, on its own, distinguish a good churn model from a worthless one, which is why accuracy is reported here only for completeness and never used as a selection criterion. (Manning, Raghavan and Schütze, 2008)

Precision is, of all the customers the model flagged as churners, the share that really did churn:

$$Precision = TP / (TP + FP)$$

It answers the business question: if the retention team acts on the model's warnings, how often is a warning a false alarm? Low precision means retention budget is spent on customers who would have stayed anyway. (Manning, Raghavan and Schütze, 2008)

Recall (sensitivity, or true positive rate) is, of all the customers who really churned, the share the model correctly flagged:

$$Recall = TP / (TP + FN)$$

In churn prediction recall is usually the more business-critical of the two, because a missed churner means a lost customer and all their future revenue, whereas a false alarm costs only a single retention offer. Precision and recall trade off against each other: flagging more customers raises recall but lowers precision, and vice versa (Manning, Raghavan and Schütze, 2008) (Fawcett (2006)).

The F1-score is the harmonic mean of precision and recall:

$$F1 = 2 \cdot (Precision \cdot Recall) / (Precision + Recall)$$

The harmonic mean is used rather than the arithmetic mean because it is dominated by the smaller of the two values: a model can only achieve a high F1 if both precision and recall are high at the same time, so it cannot hide a very low recall behind a high precision (Manning, Raghavan and Schütze, 2008).

The ROC curve (Receiver Operating Characteristic) plots the true positive rate (recall) against the false positive rate, FP / (FP + TN), tracing out every possible threshold from 0 to 1 (Fawcett, 2006). A point near the top-left corner represents a threshold that catches many churners while raising few false alarms; the diagonal represents a random classifier. The **ROC-AUC** is the area under this curve, ranging from 0.5 (random guessing) to 1.0 (perfect separation). It has a direct probabilistic interpretation: the AUC equals the probability that a randomly chosen churner is assigned a higher predicted churn probability than a randomly chosen non-churner. Because the curve is built by sweeping across all thresholds, the AUC measures the quality of the probability ranking independently of any single threshold. This matches the business goal of producing a ranked list of customers, and it is robust to the class imbalance that distorts accuracy. For both reasons, ROC-AUC is the primary metric for model selection in this project, with precision, recall and F1 used afterwards to evaluate the operating threshold (Manning, Raghavan and Schütze, 2008).

3.5 Cross Validation

To estimate how well a model generalises, the labelled data is split into a training set, used to fit the model, and a validation set, used only to measure performance on examples the model has not seen. A separate, untouched holdout set is reserved for a single final prediction. Stratified splitting preserves the class proportion in each part, which prevents random chance from placing too many churners on one side and distorting the metrics. A central risk in this process is data leakage: the situation in which information from the validation rows influences the training process, making the reported performance look better than it really is. The most common form is fitting data-dependent transformations, such as imputation of missing values or feature scaling, on the full dataset before splitting. The standard remedy is a pipeline, an object that chains the preprocessing transformers with the final estimator so that the transformation parameters are learned only from the training fold and then applied unchanged to the validation data (Pedregosa et al., 2011) (Kohavi, 1995).

To obtain reliable performance estimates , particularly during hyperparameter optimisation, where many candidate configurations must be compared ,a single train/validation split is often too dependent on which rows happen to fall on each side. k-fold cross-validation addresses this by partitioning the training data into k equally sized folds. The model is trained on $k - 1$ folds and evaluated on the remaining one, and the procedure is repeated k times so that each fold serves as the validation set exactly once. Averaging the k resulting scores yields a more robust estimate of generalisation performance than any individual split, because it no longer hinges on one arbitrary partition. For this reason cross-validation has become standard practice for model selection (Kohavi, 1995), and it is used in this project with k = 5 inside the hyperparameter search described in Section 7.4. The concrete split and pipeline used for the main train/validation separation are described in Section 5.4.

3.6 Hyperparameter Tuning

The predictive performance of a machine learning model depends not only on the available training data but also on the selection of hyperparameters. Unlike model parameters, which are automatically learned during training, hyperparameters must be defined before the learning process begins. These settings influence model complexity, training behaviour, computational efficiency, and the ability of a model to generalise to unseen data. Consequently, the selection of appropriate hyperparameter values has become an essential component of modern machine learning workflows (Bischl et al., 2023).

To understand why hyperparameter optimisation is important, it is necessary to consider the role of the individual parameters. In tree-based ensemble methods such as XGBoost and LightGBM, several hyperparameters directly influence model complexity, learning behaviour, computational efficiency and generalisation performance. They can be grouped into three categories.

The first group controls the learning process and ensemble size. The learning_rate controls how strongly each newly added tree contributes to the ensemble. Smaller values typically improve generalisation because the model learns more gradually, although a larger number of trees is then required. The n_estimators parameter specifies the total number of boosting iterations, i.e. how many trees are added to the ensemble. These two parameters interact directly, as lower learning rates generally require a larger number of estimators to achieve comparable predictive performance.

The second group controls tree complexity. The parameters num_leaves and max_depth determine how flexible the trees are allowed to become. Larger values enable the model to capture more detailed patterns in the data but also increase the risk of overfitting. The parameter min_child_samples defines the minimum number of observations required in a leaf node and therefore prevents overly specific splits based on very small groups of observations. Together, these parameters regulate the balance between underfitting and overfitting.

The third group controls randomised subsampling and regularisation. The parameters subsample and colsample_bytree introduce randomness by using only subsets of observations and features during training, which can improve generalisation and reduce variance. Finally, reg_alpha and reg_lambda correspond to L1 and L2 regularisation respectively. Both penalise overly

complex models by shrinking leaf weights and therefore reduce the risk of overfitting. Since these parameters jointly determine how complex the model is allowed to become, how strongly it is regularised and how quickly it learns, their selection has a substantial impact on predictive performance (LightGBM Developers, 2024).

The process of identifying suitable hyperparameter values is referred to as Hyperparameter Optimisation (HPO). The objective of HPO is to determine a hyperparameter configuration that maximises model performance while maintaining good generalisation ability and avoiding overfitting. According to Bischl et al. (2023), manual trial-and-error approaches are often inefficient, difficult to reproduce, and unlikely to identify optimal configurations in large search spaces.

Several approaches have been proposed for hyperparameter optimisation, including Grid Search, Random Search, Bayesian Optimisation and Evolutionary Algorithms. Grid Search systematically evaluates every possible combination of predefined hyperparameter values. For example, if three hyperparameters are assigned three candidate values each, all possible resulting configurations are tested. Although this approach is straightforward, the number of evaluations increases rapidly as the search space grows (Bergstra and Bengio, 2012).

To overcome this limitation, Random Search evaluates only a randomly selected subset of hyperparameter configurations. Bergstra and Bengio (2012) demonstrated that Random Search often achieves comparable or even superior results compared to exhaustive Grid Search while requiring substantially fewer evaluations. This is particularly beneficial when only a small subset of hyperparameters has a strong influence on model performance.

3.7 Model Interpretation and Feature Importance

A model that produces a churn score is only useful if the people acting on the scores understand why a customer was flagged. Two complementary techniques are used in this project.

Built-in feature importance in gradient-boosted tree systems such as LightGBM is based on split gain rather than on a simple count of how often a feature is used. During training, every candidate split is selected because it maximises the information gain, defined as the reduction in the loss function achieved by partitioning the data at that split (Ke et al., 2017). The importance of a feature is then obtained by summing the gains of all splits in which that feature is used. A feature is therefore regarded as important not merely because it appears in many splits, but because the splits it enables reduce the model's error substantially. The main advantage of this method is that it is computationally cheap: the gain values are a by-product of the training process and require no additional computation. Its principal limitation is that it can be biased towards high-cardinality features, that is, features with many distinct values and therefore many possible split points, which gives them more opportunities to accumulate gain irrespective of their true predictive relevance.

Permutation importance (Breiman, 2001) gives a more reliable, model-agnostic picture. For each feature in turn, its values across all validation rows are randomly shuffled. The shuffling destroys whatever relationship the feature has with the target while keeping its marginal distribution unchanged. The resulting drop in validation ROC-AUC is taken as the importance of the feature: a feature that drops the AUC sharply when shuffled is genuinely important, while one whose

shuffling leaves the AUC unchanged is essentially unused. Because it is computed from the change in a chosen performance metric rather than from the internal structure of the model, permutation importance is not affected by the high-cardinality bias of gain-based importance, although it is computationally more expensive since it requires repeated re-evaluation of the model.

Using both methods in parallel makes the interpretation more robust, because features that rank high under both can be reported to the business side with much more confidence than features that appear in only one ranking. The results are presented in Section 7.5.

4. Methodology

The methodological approach of this project follows the established workflow for supervised binary classification on tabular customer data, using the concepts defined in Section 3. The workflow is sequential, but every step was designed so that its output is reproducible and so that no information from the evaluation data leaks back into the training process.

The starting point is the formal problem definition described in Section 3.1: for each customer the model estimates a churn probability, which is then converted into a binary decision. Because retention budgets are limited, the quality of the probability ranking is treated as at least as important as the binary decision, which is why ROC-AUC (Section 3.4) is the primary selection metric. All cleaning operations were collected into a single function so that the training file and the holdout set are transformed in exactly the same way, and all data-dependent transformations were placed inside a scikit-learn pipeline to prevent the leakage discussed in Section 3.5.

The four model families of Section 3.1, Logistic Regression, Random Forest, XGBoost and LightGBM, were compared on a common validation set so that the value of the more complex methods could be measured against the simpler baselines. The moderate imbalance between churners and non-churners was addressed by the two complementary techniques of Sections 3.2 and 3.3: all four classifiers were trained with class_weight = ‘balanced’, and the decision threshold was tuned on the validation set under an explicit business constraint instead of being left at 0.50. SMOTE was included as an additional strategy for addressing class imbalance. Hyperparameter optimisation was incorporated to improve model performance, while cross-validation was used to ensure robust model evaluation. The technical principles of the tuning procedure are described in Section 3.6, and model interpretation techniques in Section 3.7.

The full analysis was implemented in Python. pandas (McKinney, 2010) and NumPy (Harris et al., 2020) were used for data handling; scikit-learn (Pedregosa et al., 2011) for preprocessing, pipelines, cross-validation and metrics; XGBoost (Chen and Guestrin, 2016) and LightGBM (Ke et al., 2017) for the boosting models; imbalanced-learn (Lemaître et al., 2017) for SMOTE; and Matplotlib (Hunter, 2007) together with seaborn (Waskom, 2021) for the figures. A fixed random seed of 42 was set wherever a stochastic operation was involved, so that every result presented in the following sections can be reproduced exactly from the same input files.

5. Dataset and Data Preparation

5.1 Dataset Description

The data used in this project is the well-known Cell2Cell dataset. It was originally compiled by the Teradata Center for Customer Relationship Management at Duke University in the United States, based on real records of a large American wireless carrier. The dataset was used in a 2002 churn prediction competition organised by the centre and has since become one of the standard benchmark datasets for customer churn research (Neslin et al., 2006). For this project the dataset was obtained from the public Kaggle page maintained by user jpacse (Kaggle, 2018).

It is important to make a distinction between the original data source and the platform on which the file is shared. Kaggle is not the original publisher of the data; it is only a hosting platform that makes the file freely available for educational and research use. The personally identifying information of customers was removed before publication, so the file contains only anonymised behavioural and contractual attributes.

The Cell2Cell file used in this work consists of two CSV files. The training file contains 51,047 customer records and 58 columns. The holdout set contains 20,000 records and the same column structure, but without the Churn label, so it can only be used for prediction, not for evaluation. The columns describe four kinds of information: how the customer uses the service (minutes of calls, dropped calls, overage minutes, percentage change in usage); how the customer is billed (monthly revenue, recurring charge, handset price); how long the customer has been with the company and how often they have contacted customer care or the retention team; and finally demographic and contractual attributes such as credit rating, age group, marital status, occupation and home-ownership. The full list of features, their types, value ranges and missing-value counts is given in Table 2.

Table 2: Features of the Cell2Cell training dataset, with type, explanation, value range and missing-value statistics.

#	Feature	Type	Explanation	Values / Range	Missing	Miss %
0	CustomerID	metric	Unique customer identifier; not used as predictive feature	3000002–3399994	0	0.0
1	Churn	nominal	Target variable: whether the customer cancelled the contract	No, Yes	0	0.0
2	MonthlyRevenue	metric	Average monthly revenue generated by the customer	−6.17 – 1223.38	156	0.31
3	MonthlyMinutes	metric	Average number of monthly call minutes	0.0 – 7359.0	156	0.31
4	TotalRecurringCharge	metric	Monthly recurring charge paid by the customer	−11.0 – 400.0	156	0.31
5	DirectorAssistedCalls	metric	Number of calls assisted by an operator	0.0 – 159.39	156	0.31
6	OverageMinutes	metric	Minutes used beyond the included plan allowance	0.0 – 4321.0	156	0.31
7	RoamingCalls	metric	Number of roaming calls	0.0 – 1112.4	156	0.31
8	PercChangeMinutes	metric	Percentage change in monthly minutes	−3875.0 – 5192.0	367	0.72
9	PercChangeRevenues	metric	Percentage change in monthly revenue	−1107.7 – 2483.5	367	0.72
10	DroppedCalls	metric	Number of dropped calls	0.0 – 221.7	0	0.0
11	BlockedCalls	metric	Number of blocked calls	0.0 – 384.3	0	0.0
12	UnansweredCalls	metric	Number of unanswered calls	0.0 – 848.7	0	0.0
13	CustomerCareCalls	metric	Number of calls to customer care	0.0 – 327.3	0	0.0
14	ThreewayCalls	metric	Number of three-way calls	0.0 – 66.0	0	0.0
15	ReceivedCalls	metric	Number of received calls	0.0 – 2692.4	0	0.0

#	Feature	Type	Explanation	Values / Range	Missing	Miss %
16	OutboundCalls	metric	Number of outgoing calls	0.0 – 644.3	0	0.0
17	InboundCalls	metric	Number of incoming calls	0.0 – 519.3	0	0.0
18	PeakCallsInOut	metric	Number of calls during peak hours	0.0 – 2090.7	0	0.0
19	OffPeakCallsInOut	metric	Number of calls during off-peak hours	0.0 – 1474.7	0	0.0
20	DroppedBlockedCalls	metric	Combined number of dropped and blocked calls	0.0 – 411.7	0	0.0
21	CallForwardingCalls	metric	Number of call forwarding calls	0.0 – 81.3	0	0.0
22	CallWaitingCalls	metric	Number of call waiting calls	0.0 – 212.7	0	0.0
23	MonthsInService	metric	Number of months the customer has been with the provider	6 – 61	0	0.0
24	UniqueSubs	metric	Number of unique subscribers linked to the account	1 – 196	0	0.0
25	ActiveSubs	metric	Number of active subscribers linked to the account	0 – 53	0	0.0
26	ServiceArea	nominal	Customer service area or region	AIRAIK803, AIRAND864, …	24	0.05
27	Handsets	metric	Number of handsets used by the customer	1.0 – 24.0	1	0.0
28	HandsetModels	metric	Number of different handset models used	1.0 – 15.0	1	0.0
29	CurrentEquipmentDays	metric	Number of days the current equipment has been in use	−5.0 – 1812.0	1	0.0
30	AgeHH1	metric	Age of the first household member	0.0 – 99.0	909	1.78
31	AgeHH2	metric	Age of the second household member	0.0 – 99.0	909	1.78
32	ChildrenInHH	nominal	Whether children live in the household	No, Yes	0	0.0
33	HandsetRefurbished	nominal	Whether the handset is refurbished	No, Yes	0	0.0
34	HandsetWebCapable	nominal	Whether the handset supports web access	No, Yes	0	0.0
35	TruckOwner	nominal	Whether the customer owns a truck	No, Yes	0	0.0
36	RVOwner	nominal	Whether the customer owns a recreational vehicle	No, Yes	0	0.0
37	Homeownership	nominal	Customer home ownership status	Known, Unknown	0	0.0
38	BuysViaMailOrder	nominal	Whether the customer buys via mail order	No, Yes	0	0.0
39	RespondsToMailOffers	nominal	Whether the customer responds to mail offers	No, Yes	0	0.0
40	OptOutMailings	nominal	Whether the customer opted out of mailings	No, Yes	0	0.0
41	NonUSTravel	nominal	Whether the customer travels outside the US	No, Yes	0	0.0
42	OwnsComputer	nominal	Whether the customer owns a computer	No, Yes	0	0.0
43	HasCreditCard	nominal	Whether the customer has a credit card	No, Yes	0	0.0
44	RetentionCalls	metric	Number of retention-related calls	0 – 4	0	0.0
45	RetentionOffersAccepted	metric	Number of accepted retention offers	0 – 3	0	0.0
46	NewCellphoneUser	nominal	Whether the customer is a new cellphone user	No, Yes	0	0.0
47	NotNewCellphoneUser	nominal	Whether the customer is not a new cellphone user	No, Yes	0	0.0
48	ReferralsMadeBySubscriber	metric	Number of referrals made by the customer	0 – 35	0	0.0
49	IncomeGroup	metric	Income group of the customer	0 – 9	0	0.0
50	OwnsMotorcycle	nominal	Whether the customer owns a motorcycle	No, Yes	0	0.0
51	AdjustmentsToCreditRating	metric	Number of adjustments made to credit rating	0 – 25	0	0.0
52	HandsetPrice	nominal	Price of the customer handset	10, 100, 130, 150, 180, 200, …	0	0.0
53	MadeCallToRetentionTeam	nominal	Whether the customer contacted the retention team	No, Yes	0	0.0
54	CreditRating	nominal	Customer credit rating category	1-Highest … 7-Lowest	0	0.0
55	PrizmCode	nominal	Customer lifestyle / market segmentation group	Other, Rural, Suburban, Town	0	0.0
56	Occupation	nominal	Customer occupation category	Clerical, Crafts, Homemaker, …	0	0.0
57	MaritalStatus	nominal	Customer marital status	No, Unknown, Yes	0	0.0

The target variable is the column Churn. It takes the value Yes if the customer left during the observation window and No if they stayed. In the training set, 28.8% of the customers are churners and 71.2% are non-churners. This is a moderate class imbalance and has to be taken into account when choosing the evaluation metrics and the training strategy, as discussed in Sections 3.2 and 3.4.

The Cell2Cell dataset has clear strengths and weaknesses for a study of this kind. On the positive side, it is large (over 70,000 records in total), it has been used in a published benchmark study so that results can be put in context with the wider literature, it covers a wide range of behavioural, contractual and demographic attributes in a single file, and it provides a separate unlabelled holdout set which forces a clean separation between development and final evaluation. On the negative side, the data originates from the early 2000s and therefore reflects a tariff and device landscape that no longer exists in the same form; each customer is described by only one static snapshot, which rules out time-series or survival modelling; the geographical column ServiceArea contains more than 700 unique area codes, and although these could in principle be grouped into broader regions through feature engineering, this was outside the scope of the current project and the column was excluded; and the dataset is moderately imbalanced. These limitations are accepted here and returned to in Section 9.

5.2 Data Understanding and Cleaning

Before any model is built, the data has to be understood. Skipping this step is one of the most common mistakes in applied machine learning: cleaning or modelling a dataset that has not been properly inspected often leads to silent errors that only show up much later, when the predictions produce unexpected results and the source of the discrepancy is not immediately apparent.

The first inspection of the training file showed 51,047 rows and 58 columns, with no duplicated rows. Of the 58 columns, 36 are numeric (call counts, durations, charges, ages) and 22 are categorical, mainly binary Yes/No flags such as HasCreditCard or OwnsComputer, but also a few multi-level categories such as Occupation, MaritalStatus, PrizmCode and CreditRating. The column CustomerID is technically numeric but is in fact only a unique identifier; it carries no predictive information and was removed before modelling.

A few irregularities were detected during inspection. The column HandsetPrice was stored as text rather than as a number because it contained the string “Unknown” alongside the numeric prices. The column CreditRating used codes such as 1-Highest, 2-High, …, 7-Lowest, which form an ordered scale but had to be converted into a real numeric rank before they could be used by a model. The column ServiceArea contained more than 700 unique area codes; such a high number of categories carries little useful information and would cause every model to overfit to specific codes, so this column was removed during cleaning.

Cleaning the data means fixing every quality issue that would otherwise mislead the model. The cleaning steps were collected into a single function so that exactly the same operations could later be applied to the holdout set. Reusing one function for both files is important: if the holdout were cleaned differently, the model would see columns or values it was never trained on, and predictions would silently degrade.

First, the identifier column CustomerID was dropped because it has no predictive meaning. **Second**, the target column Churn was encoded into binary form (Yes → 1, No → 0). **Third**, the column HandsetPrice was repaired by replacing the string "Unknown" with a proper missing value (NA) and then casting the column to floating-point numbers. **Fourth**, the ordered codes in CreditRating were converted into a numeric rank from 1 (best credit) to 7 (worst credit) by extracting the digit at the beginning of the string. **Finally**, the redundant binary column NotNewCellphoneUser, which is just the logical inverse of NewCellphoneUser, was removed, together with the high-cardinality column ServiceArea. After these operations the dataset had 51,047 rows and 55 columns.

The dataset also contained values that were not technically missing but appeared physically unrealistic. Three rows contained negative values in MonthlyRevenue, and 76 rows contained negative values in CurrentEquipmentDays. Negative values in CurrentEquipmentDays are not logically meaningful, because a device cannot be used for a negative number of days; negative MonthlyRevenue values may occur in special billing situations such as refunds, credits or accounting adjustments.

These observations were left in the dataset for two reasons. First, the final model is based on gradient boosting trees, which are generally robust to a small number of unusual extreme values because they split data using decision thresholds rather than relying on distance calculations. Second, the number of affected rows was very small relative to the full dataset, so removing or modifying them would provide little practical benefit while introducing additional preprocessing rules that would also have to be applied consistently to the holdout set.

It is also worth noting why ordinary statistical outliers were not removed. Telecom usage data is naturally right-skewed: most customers use the service moderately, while a small number of heavy users produce very large values. The interquartile-range rule, which is designed for symmetric distributions, would flag many of these heavy-user values as outliers, but they are not data errors; they are exactly the kind of customer behaviour that is most relevant for churn prediction. Removing such rows would destroy real signal.

5.3 Missing Value Treatment

Several columns contained missing values. A missing value is simply an empty cell where the system did not record any information. In the training set 14 columns had missing values, but in every case the proportion was below 5%. The columns with the most gaps were AgeHH1 and AgeHH2 (each around 909 missing rows, roughly 1.8%) and PercChangeMinutes and PercChangeRevenues (around 367 rows each).

Missing values were deliberately not filled during cleaning. Filling them before the train/validation split would let information from the validation rows influence the imputation values used during training, the data leakage explained in Section 3.5. The imputation was therefore moved into the preprocessing pipeline, where it is fitted only on the training portion. Because the proportion of missing values is small, the affected rows do not need to be dropped; the gaps can be filled in (imputed) as part of the preprocessing pipeline described in Section 5.4.

5.4 Feature Engineering and Imputation

Machine learning models cannot work on raw text and they cannot work on data that contains gaps, and numeric features work better when they are scaled to a similar range. To handle all of this in one place, the transformations were collected into a scikit-learn pipeline, the object introduced in Section 3.5 that chains several transformation steps and a final model together. Calling fit on the pipeline applies all transformations in order, and calling predict applies the same transformations to new data.

The preprocessing pipeline has two branches. The numeric branch first fills missing values with the median of the column and then applies a StandardScaler, which subtracts the mean and divides by the standard deviation so that every feature has roughly the same scale. The median was chosen over the mean because telecom data is skewed and the median is more robust against the heavy right tail. A more sophisticated alternative would be Random Forest imputation (MissForest), which uses the relationships between features to predict missing values; however, since the proportion of missing values is below 2% in all affected columns, the simpler median imputation is sufficient and the impact on model performance is negligible at this scale of missingness, particularly when the proportion of missing values is very low (Little and Rubin, 2019). The categorical branch first fills missing values with the constant string “missing”, which is then treated as its own category, and finally encodes each category as an integer using an OrdinalEncoder. The encoder is configured with handle_unknown = ‘use_encoded_value’ and unknown_value = −1, so that any new category appearing only in the holdout set does not cause the prediction to crash.

As explained in Section 3.5, placing all data-dependent transformations inside the pipeline ensures that no information from the validation data leaks into the training process: the imputer and the scaler are fitted only on the training portion of each fold.

Following the validation strategy described in Section 3.5, the cleaned training file of 51,047 customers is split into two parts. The training part (80%, 40,837 rows) is used to fit the models and the validation part (20%, 10,210 rows) is used to measure how well the models generalise to data they have never seen. The split was carried out with sklearn.model_selection.train_test_split and a fixed random seed (random_state = 42). The argument **stratify = y** preserves the proportion of churners in both halves; without it, random chance could place too many churners on one side and distort the validation metrics. With stratification, both halves contain approximately 28.8% churners. The separate holdout set (20,000 customers) was kept completely untouched during the entire training, tuning and threshold-selection process, and was used only at the very end to generate the final predictions.

6. Exploratory Data Analysis and Feature Selection

Exploratory data analysis (EDA) means looking at the data with simple statistics and plots before any modelling. Its purpose is to understand which features carry useful information about churn, which features are redundant with each other, and how strong the class imbalance really is. This stage directly informs the modelling choices that follow.

Churn Rate by Category

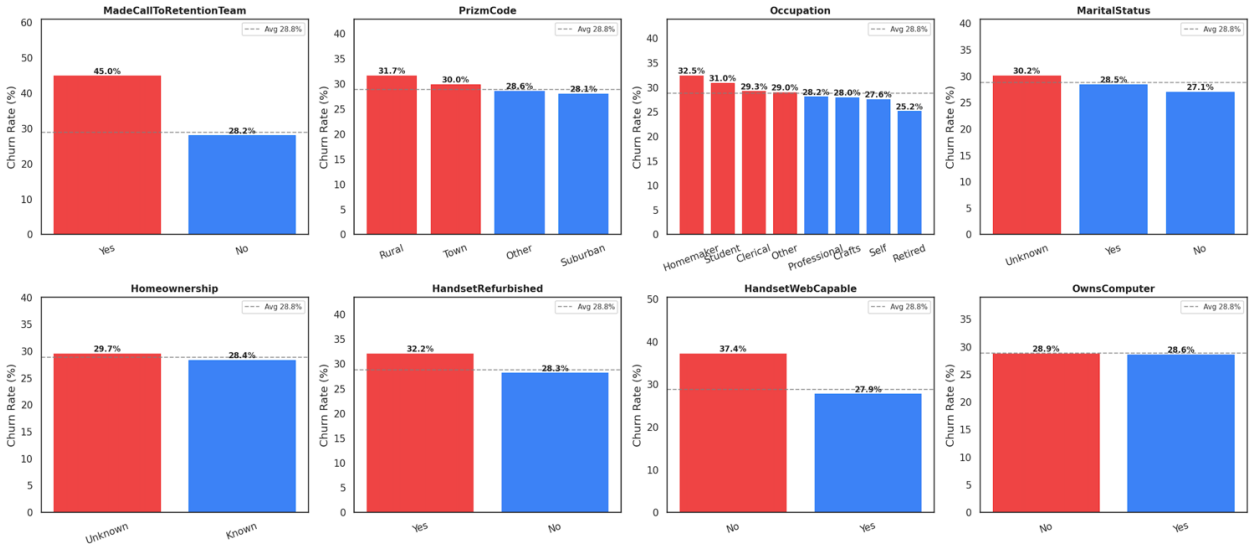


Figure 1: Churn rate by selected categorical features, compared to the overall churn rate.

Figure 1 compares the churn rate of customers in each category against the overall average of 28.8%. Two patterns stand out clearly. First, customers who already made a call to the retention team (MadeCallToRetentionTeam = Yes) churn at 45.0%, which is more than 16 percentage points above the average, by far the strongest categorical signal in the data. Second, customers whose handset is not web-capable (HandsetWebCapable = No) churn at 37.4%, roughly 8.6 percentage points above average, suggesting that outdated device capability is a meaningful risk factor. Customers with a refurbished handset (HandsetRefurbished = Yes) also show a moderately elevated churn rate of 32.2%. The remaining categorical features, PrizmCode, Occupation, MaritalStatus, Homeownership and OwnsComputer, show differences of less than 2 percentage points across their categories, which is too small to be considered meaningful and is likely due to chance. These early findings already give the retention team a first idea of high-risk groups, even before any model is trained.

Numeric Features by Churn Status

Figure 2 overlays the distributions of churners and non-churners for several key numeric features. To improve readability, all axes are capped at the 99th percentile of each feature, meaning the top 1% of extreme values are excluded from the plot. This is necessary because telecom usage data is heavily right-skewed: a small number of customers produce extreme values that would otherwise stretch the x-axis and compress the distribution of the remaining 99% of customers into an unreadable narrow band. The capping applies only to the visualisation, the underlying data rows

are not removed or modified, and the model uses the full original values. The clearest separation appears for CurrentEquipmentDays: churners tend to have noticeably older phones than non-churners. The same direction is visible for CustomerCareCalls, where churners on average contact customer support more often. MonthsInService shows the opposite effect: long-tenure customers churn less. TotalRecurringCharge also shows a visible difference, with churners tending to have slightly higher recurring charges, suggesting that pricing pressure contributes to the decision to leave. For OverageMinutes, the distributions overlap heavily and no clear separation is visible, indicating that overage usage alone is not a strong churn signal. The overall interpretation is that device age, complaint frequency, tenure and recurring charge together capture a large part of the signal in the data.

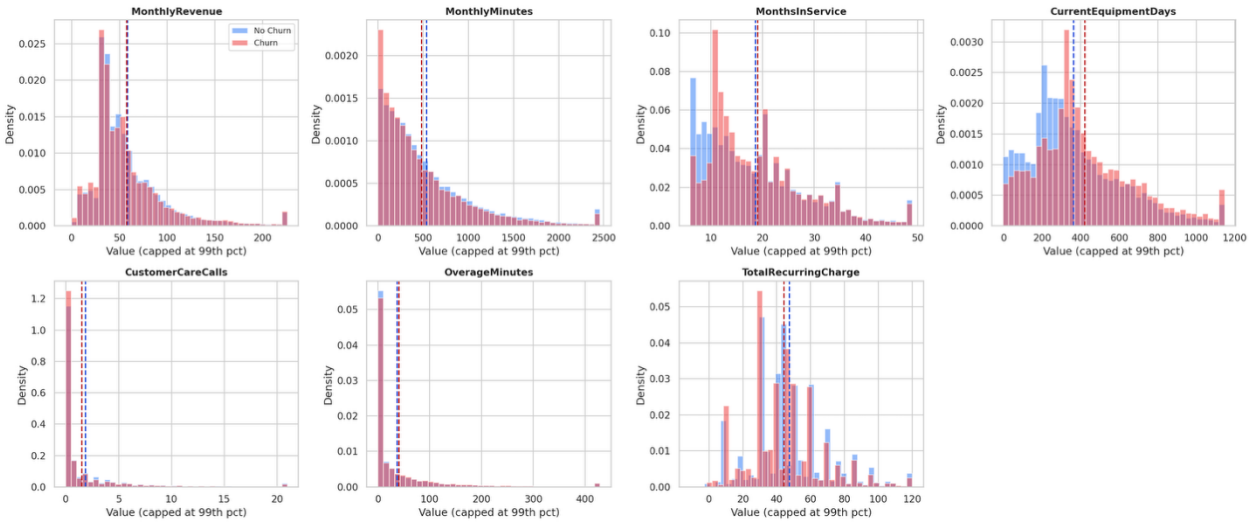


Figure 2: Histograms of selected numeric features, split into churners and non-churners.

Correlation and Feature Selection

Figure 3 shows the pairwise Bravais-Pearson correlation coefficient of all numeric features. Two observations stand out. First, no individual feature is strongly correlated with the target Churn; the strongest single correlation is +0.10 for CurrentEquipmentDays, followed by RetentionCalls and TotalRecurringCharge at around 0.06. This means churn is multi-factorial: it does not come from one single feature but from many weak signals combined. Second, several pairs of features are highly correlated with each other ($|r| > 0.7$), for example Handsets and HandsetModels ($r = 0.89$) or MonthlyMinutes and ReceivedCalls ($r = 0.83$). For each redundant pair, the feature less correlated with Churn was removed. This is a simple but pragmatic approach; a more principled alternative would be a criterion that explicitly accounts for multicollinearity, such as mutual information, which measures how much knowing one feature reduces the uncertainty about another and, unlike the Pearson correlation used here, also captures non-linear dependencies. This rule eliminated ten features: Handsets, ReceivedCalls, BlockedCalls, ActiveSubs, PeakCallsInOut, OffPeakCallsInOut, OutboundCalls, RetentionOffersAccepted, UnansweredCalls and DroppedBlockedCalls.

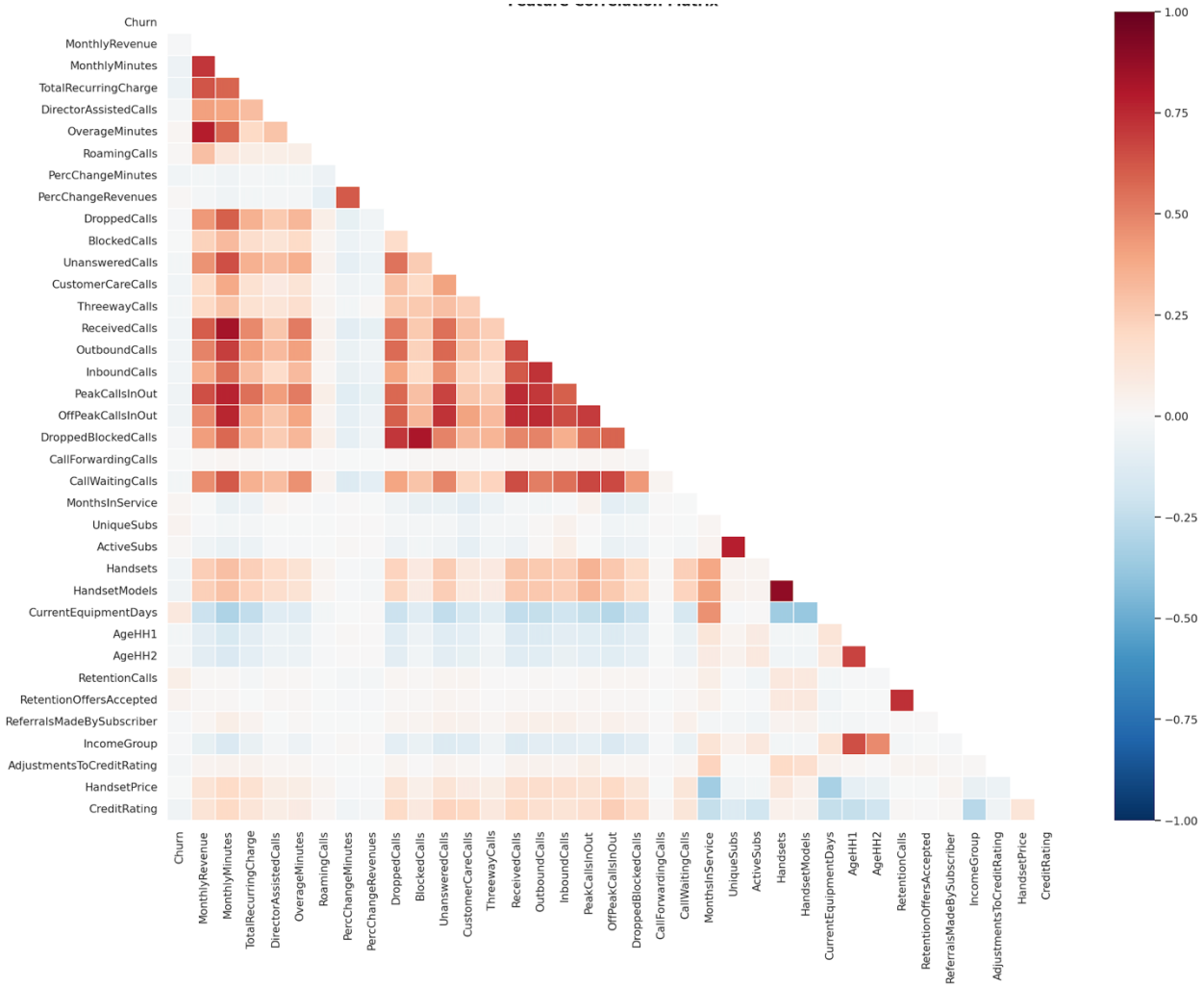


Figure 3: Correlation matrix of the numeric features.

Device Age and Other Strong Drivers

Figure 4 looks at one of the most important features more closely. Customers with very old equipment churn at a higher rate than customers with recent devices. This is consistent with the wider literature: an old phone is a natural moment for a customer to look at competing offers. The variables that appear most important on the basis of the EDA are CurrentEquipmentDays, CustomerCareCalls, MonthsInService, MonthlyRevenue and TotalRecurringCharge. The same features later appear at the top of the model interpretation results in Section 7.5, which confirms that the EDA findings are consistent with what the model learns on its own.

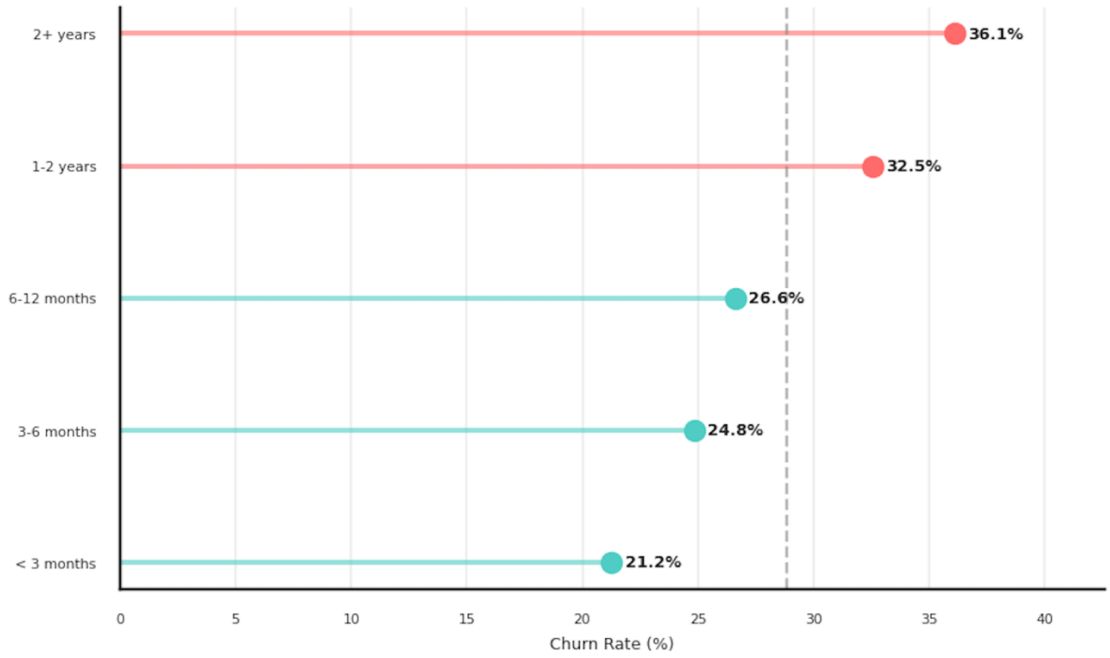


Figure 4: Churn rate by device age group (CurrentEquipmentDays), grouped into five brackets. Red bars indicate above-average churn rate.

7. Modelling and Results

7.1 Baseline Model Comparison

A baseline model is a deliberately simple model whose only purpose is to give a reference point: if a more complex model does not clearly beat the baseline, the extra complexity is not justified. In an imbalanced problem the natural baseline is a classifier that always predicts the majority class "No Churn". This baseline reaches 71.2% accuracy but identifies zero churners, which confirms the argument of Section 3.4 that accuracy on its own is not a useful metric for this dataset.

The four model families described in Section 3.1, Logistic Regression, Random Forest, XGBoost and LightGBM, were then trained on the same training set and evaluated on the same validation set, all using the class_weight = 'balanced' option of Section 3.2 and the default decision threshold of 0.50. The models were trained on the feature set obtained after the correlation-based feature selection of Section 6.3. An informal check against the full feature set showed no measurable change in validation ROC-AUC, confirming that the eleven removed features carried no information that the remaining ones did not already cover. The validation metrics are reported in Table 3.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.5918	0.3704	0.5955	0.4567	0.6248
Random Forest	0.6397	0.4055	0.5374	0.4622	0.6586
XGBoost	0.6404	0.4138	0.5955	0.4883	0.6772
LightGBM	0.6272	0.4046	0.6227	0.4905	0.6765

Table 3: Validation metrics for the four candidate models at the default threshold of 0.50.

ROC Models curves

Figure 5 plots the ROC curves of the four models. The two gradient boosting models, XGBoost (AUC = 0.6772) and LightGBM (AUC = 0.6765), perform almost identically and clearly above Random Forest (0.6586) and Logistic Regression (0.6248). The dashed diagonal corresponds to a random classifier. The interpretation is that XGBoost and LightGBM combine the many weak signals in the data better than the linear model and slightly better than a plain Random Forest. Either of the two gradient boosting models is a reasonable candidate for further tuning. This answers RQ1: gradient boosting is the strongest family on this dataset, but its lead over Random Forest is modest and the two boosting models are effectively tied.

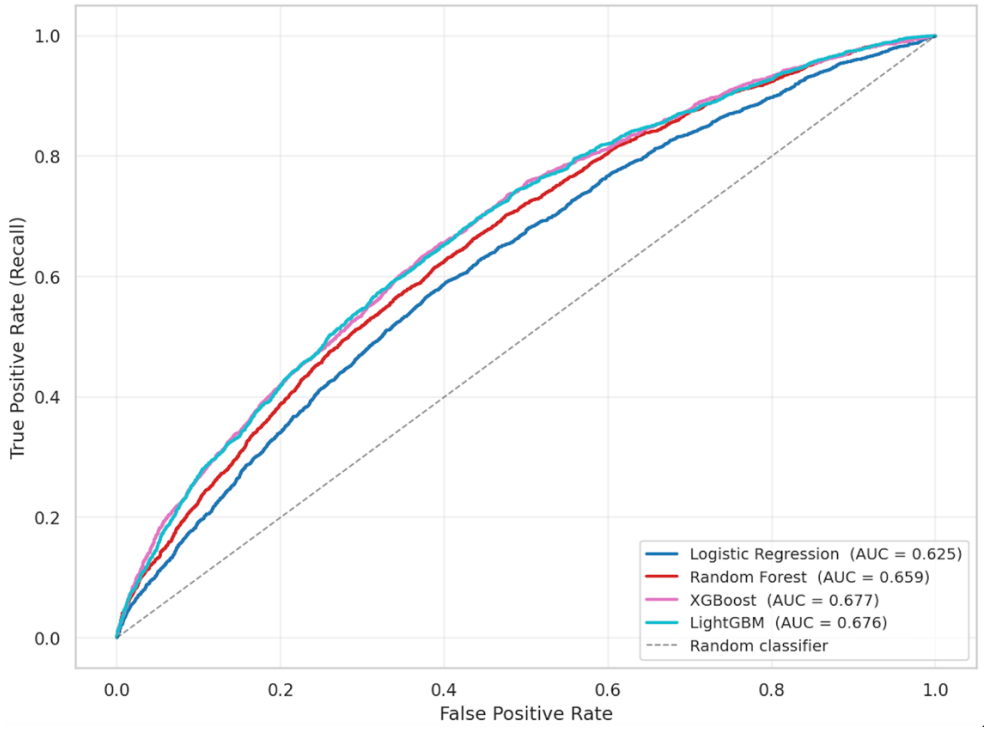


Figure 5: ROC curves for Logistic Regression, Random Forest, XGBoost and LightGBM.

7.2 Threshold Optimisation

As explained in Section 3.4, every metric except ROC-AUC depends on a fixed decision threshold, the probability cut-off above which a customer is labelled a churner. The default of 0.50 is convenient but not necessarily the best choice: when the classes are imbalanced, models tend to be too conservative at 0.50 and miss many real churners. Lowering the threshold flags more customers as risky, catching more real leavers (higher recall) at the cost of more false alarms (lower precision).

The right threshold is therefore a business decision rather than a purely mathematical one. The cost of missing a real churner is the lost customer; the cost of a false alarm is one retention offer wasted on someone who would have stayed anyway. Because keeping a customer is much more valuable than the cost of a single retention call, it usually makes sense to lower the threshold below 0.50.

Figure 6 shows how Precision, Recall and F1 change with the threshold for the best base model by validation AUC (XGBoost), evaluated on the validation set. The threshold was deliberately tuned on the validation set rather than the training set, because measuring it on the training data would give an optimistically biased cut-off that would not transfer to unseen customers. At a threshold of 0.50 the model achieves Precision = 0.4138, Recall = 0.5955 and F1 = 0.4883 with 41.5% of customers flagged. **A business rule was then applied**: only thresholds with Recall ≥ 60% and at most 45% of customers flagged were considered acceptable. Under this rule the best F1 was reached at a threshold of 0.49, which yields Precision = 0.4078, Recall = 0.6258, F1 = 0.4938 and 44.2% of customers flagged. Lowering the threshold by only one percentage point

therefore raised Recall by about three points without flagging an unreasonable share of the customer base, and the threshold 0.49 was used for all later evaluations.

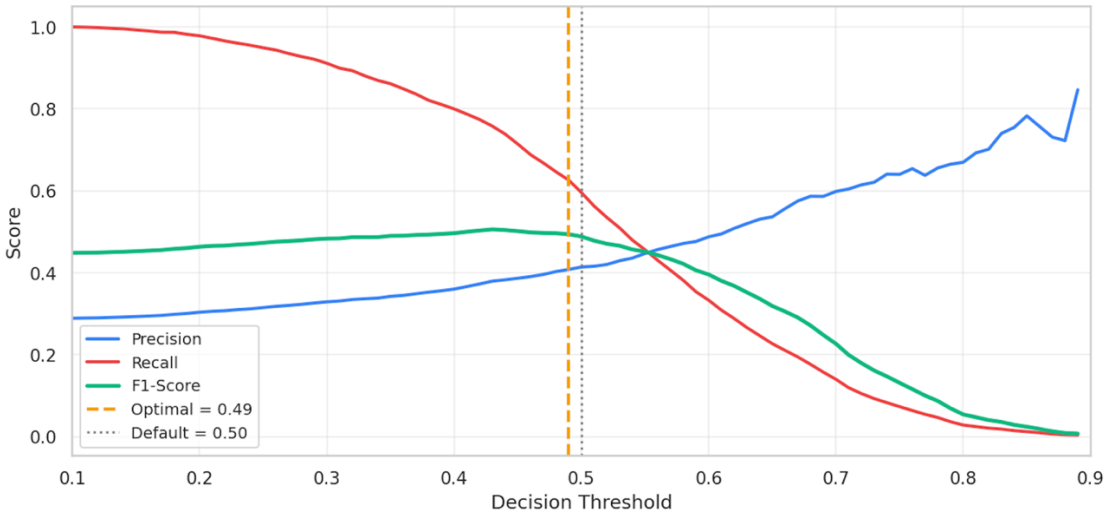


Figure 6: Precision, Recall and F1-score of the best model across decision thresholds from 0.10 to 0.90.

7.3 SMOTE Evaluation

As described in Section 3.3, SMOTE is the second common strategy for class imbalance, generating synthetic minority examples instead of reweighting the existing ones. It was tested here to assess whether it adds anything to the class-weighting and threshold tuning already in place. SMOTE was applied to the training data only; applying it to the validation data would have made the evaluation meaningless, because the model would then be evaluated on fake examples it had partly seen during training. After SMOTE the training set grew from 40,837 rows (29,068 non-churners and 11,769 churners) to 58,136 rows (29,068 non-churners and 29,068 synthetic-balanced churners). A LightGBM model was then trained on the SMOTE-balanced data and evaluated on the original validation set.

The result was that SMOTE did not improve performance. The best baseline model without SMOTE (XGBoost) reached ROC-AUC = 0.6772 on the validation set, while the SMOTE-trained LightGBM reached ROC-AUC = 0.6740, a small loss of about 0.0032 points. SMOTE did push the accuracy upwards by being more selective with its churn predictions, but recall dropped sharply (to around 0.15–0.17 at threshold 0.50), which is the opposite of what is wanted in this problem. The reason recall collapses while the AUC barely moves is that SMOTE changes the calibration of the predicted probabilities: the same 0.50 threshold becomes far more conservative on a SMOTE-trained model than on a class-weighted one, so fewer customers cross it even though the underlying ranking quality, which AUC measures, is almost unchanged. The project therefore continued without SMOTE and relied on class weighting plus threshold tuning instead. This outcome is consistent with the literature (López et al., 2013): when the imbalance is only moderate and a gradient boosting model with balanced class weights is already used, SMOTE usually adds little and sometimes hurts. Together with Section 7.2, this answers RQ2: the most useful trade-off

is achieved by class weighting combined with a tuned threshold, while synthetic oversampling adds no value on this dataset.

7.4 Hyperparameter Tuning

Following the procedure described in Section 3.6, RandomizedSearchCV with five-fold cross-validation was used to tune LightGBM. The search explored a wide range of values for learning_rate, max_depth, num_leaves, n_estimators, min_child_samples, subsample, colsample_bytree, reg_alpha and reg_lambda. The best cross-validation ROC-AUC was 0.6797. The best parameter combination was learning_rate ≈ 0.133, max_depth = 3, num_leaves = 8, n_estimators = 332, min_child_samples = 22, subsample ≈ 0.77, colsample_bytree ≈ 0.64, reg_alpha ≈ 0.007 and reg_lambda ≈ 0.51. Applied on the held-out validation set with the optimal threshold of 0.49, the tuned model reached ROC-AUC = 0.6745, Recall = 0.6526, F1 = 0.4919 and Accuracy = 0.6116.

It is important to read these numbers honestly. Tuning improved the cross-validation score (from roughly 0.677 to 0.680), but on the separately held validation set the improvement over the untuned LightGBM was very small, and the recall rose at the cost of accuracy. A separate focused GridSearch on XGBoost (64 candidates, five-fold cross-validation) achieved a slightly higher validation AUC of 0.6778. This is a typical pattern, consistent with the literature reviewed in Section 2: most of the predictive power comes from the data itself and from the choice of model family, while hyperparameter tuning produces only a small additional gain. Tuning is still worthwhile because it reduces the risk of selecting a clearly suboptimal configuration, but it should not be expected to fix a fundamentally hard problem.

A note on the experimental design is in order. The baseline comparison in Section 7.1 left XGBoost and LightGBM essentially tied, with a difference in validation AUC far smaller than the variation expected from a single train/validation split. Because the two boosting models were statistically interchangeable on this dataset, the subsequent experiments did not commit prematurely to one of them: the decision threshold was located on XGBoost in Section 7.2, the SMOTE experiment was carried out on LightGBM in Section 7.3, and the randomised hyperparameter search in this section was run on LightGBM, complemented by the focused GridSearch on XGBoost reported above. This is justified because gradient-boosted decision trees share the same split-based decision logic, so a threshold obtained on one transfers cleanly to the other; the numbers above confirm that the 0.49 threshold derived from XGBoost still produces a recall of about 0.65 on the tuned LightGBM. The final choice of LightGBM over XGBoost as the deployed model is therefore made on practical grounds, primarily its faster training time, rather than on a statistically meaningful AUC advantage.

Finally, a soft-voting ensemble combining the three tree-based models (the tuned LightGBM, the tuned XGBoost and Random Forest) was evaluated as a further candidate. The ensemble averages the predicted churn probabilities of its members, with the expectation that errors of the individual models partially cancel out. On the validation set the ensemble reached a ROC-AUC of 0.6778, marginally above the tuned LightGBM (0.6745) and identical to the tuned XGBoost ,but at the selected threshold of 0.49 its recall dropped to 0.6241, against 0.6526 for the tuned LightGBM. Since recall is the business-critical metric in this project (Section 3.4), the small gain in ranking

quality does not compensate for missing more actual churners, and the ensemble was not adopted as the final model.

7.5 Model Interpretation

As motivated in Section 3.7, the final model was interpreted with two complementary techniques, built-in feature importance and permutation importance, so that the drivers reported to the business side could be cross-checked against each other.

7.5.1 Built-in Feature Importance

Recall from Section 3.7 that built-in importance in LightGBM is based on split gain, the total reduction in the loss function (information gain) contributed by all splits in which a feature is used , rather than on a simple count of splits, and that its main limitation is a bias towards high-cardinality features. Figure 7 shows the resulting ranking for the tuned LightGBM model. The five red bars mark the five most important drivers and the blue bars show the remaining features in the top 25, ordered by their total split gain.

The ranking is consistent with the exploratory analysis. The strongest driver, CurrentEquipmentDays, was already the clearest separator between churners and non-churners in Figure 2 and carried the highest single correlation with the target in Figure 3. MadeCallToRetentionTeam and RetentionCalls, which rank second and fifth, correspond to the strongest categorical signal identified in Figure 1, where customers who had contacted the retention team churned at 45.0% against a 28.8% average. MonthsInService reflects the tenure effect seen in Figure 2, and HandsetRefurbished matches the elevated churn rate of refurbished-handset customers observed during EDA. The agreement between the unsupervised exploratory findings and the model's internal importance increases confidence that the model is learning genuine churn patterns rather than artefacts of the data.

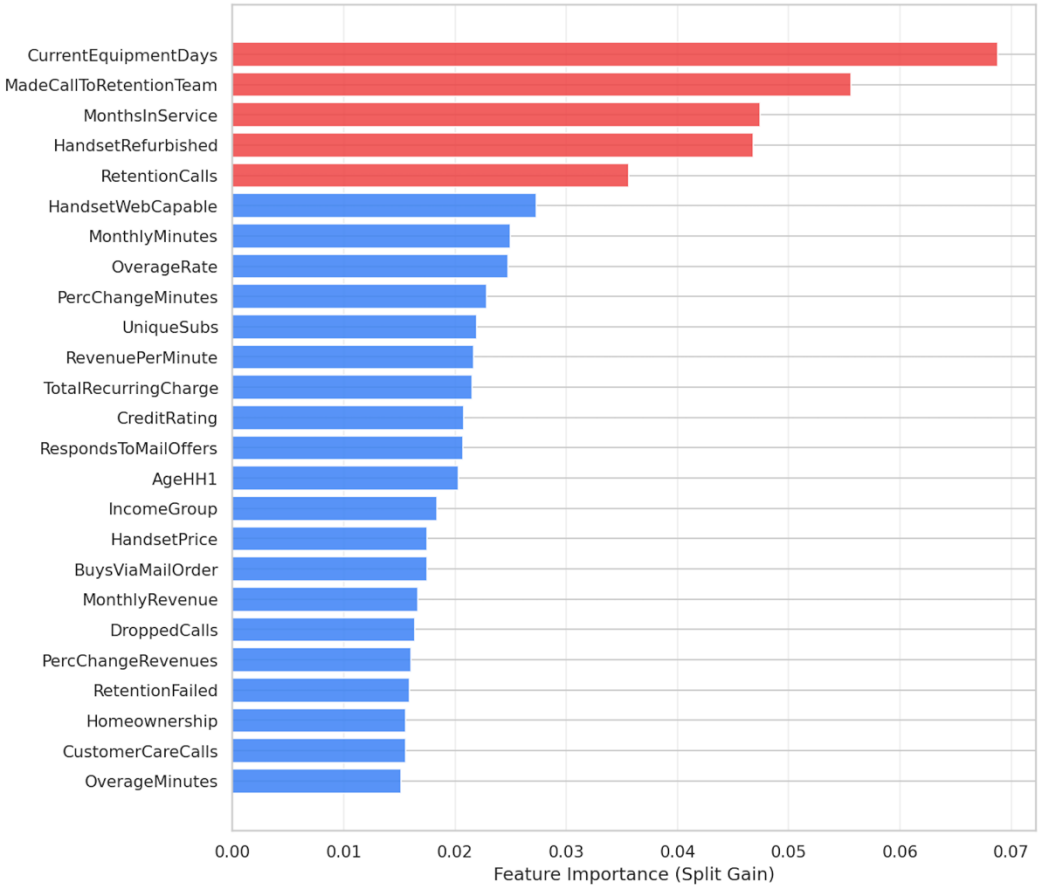


Figure 7: Top features by built-in LightGBM feature importance (split gain). The five red bars mark the most important drivers; the blue bars show the remaining features in the top 25.

7.5.2 Permutation Importance

Figure 8 shows the ten features with the largest AUC drop on the validation set, averaged over ten repetitions of the shuffling procedure. The error bars show the standard deviation across these repeats. Short error bars indicate that the importance estimate is stable and not strongly affected by the randomness of the shuffling process. CurrentEquipmentDays and MonthsInService clearly dominate the ranking, confirming the EDA findings that device age and customer tenure are the strongest churn indicators.

Comparing Figures 7 and 8 shows strong agreement between built-in gain importance and permutation importance. CurrentEquipmentDays and MonthsInService rank among the most important features in both methods, which increases confidence that they are genuine drivers of churn. Some features change position because built-in gain importance reflects the contribution of a feature during tree construction, whereas permutation importance measures its actual effect on validation performance. Together, the two methods answer RQ3 by identifying equipment age, tenure, recent usage change and recurring charge as the dominant and method-stable drivers.

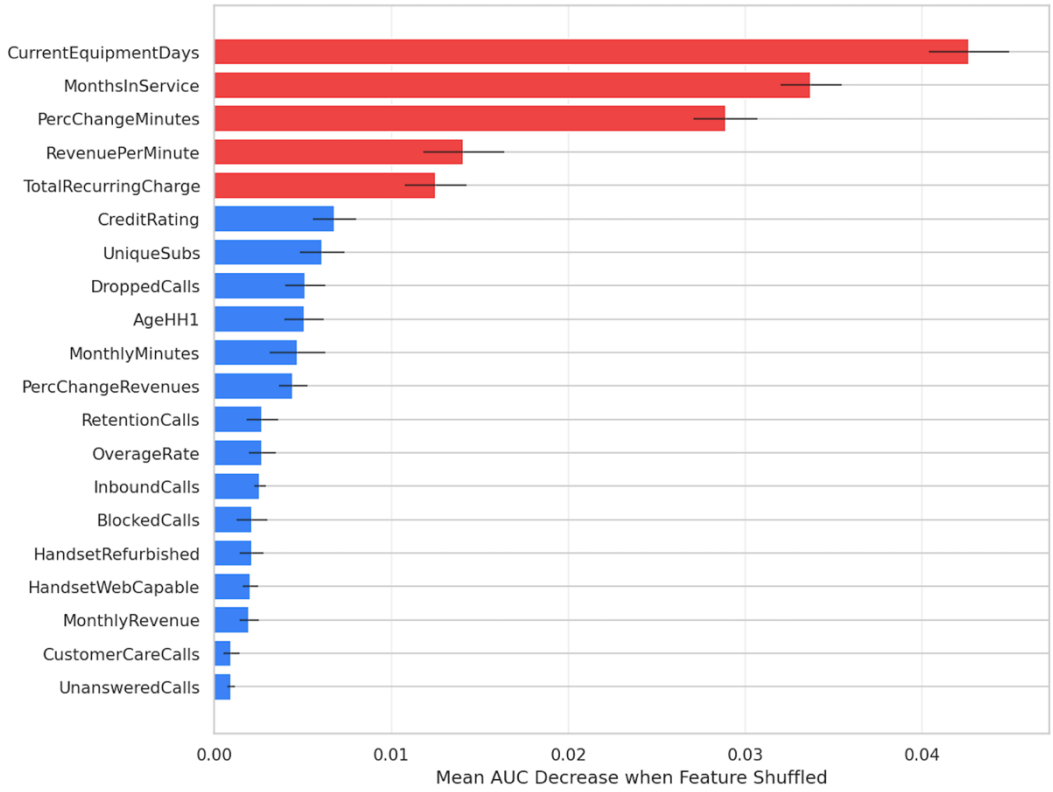


Figure 8: Top features by permutation importance (mean ROC-AUC drop after shuffling, with error bars showing the standard deviation over ten repeats). The red bars mark the most important drivers

7.6 Final Model Performance

The tuned LightGBM model with class_weight = 'balanced' and a decision threshold of 0.49 was selected as the final model. The reason for this choice is a combination of three factors. First, LightGBM was, together with XGBoost, the best-performing model in the baseline comparison of Section 7.1. Second, gradient boosting trees handle the natural skew and the impossible-but-rare values in telecom data without additional preprocessing. Third, because the tuned XGBoost, the tuned LightGBM and the voting ensemble of Section 7.4 were effectively tied on validation AUC (0.6778, 0.6745 and 0.6778 respectively, differences within the noise of a single split), while the ensemble additionally suffered the lowest recall, the choice was made on practical grounds: LightGBM trains faster on this dataset, approximately 3.3 seconds versus 4.4 seconds for XGBoost on the same hardware, a speed-up of about 1.3×, which makes it more convenient for periodic retraining on fresh data in production.

On the held-out validation set, the final model achieves Accuracy = 0.6116, Precision = 0.3947, Recall = 0.6526, F1 = 0.4919 and ROC-AUC = 0.6745. When the same trained pipeline is applied to the 20,000 customers in the holdout set, it flags 9,688 customers (48.4%) as churn candidates. The distribution of predicted probabilities on the holdout set is almost identical to the distribution on the validation set (mean probability 0.4738 against 0.4728), which is reassuring: the model behaves similarly on data it has never seen, with no obvious sign of overfitting or distribution shift.

These numbers need to be interpreted honestly. An ROC-AUC of about 0.67 corresponds to moderate predictive power. The model is much better than random guessing (AUC = 0.5) and

much better than a constant prediction of “No Churn”, but it is far from a perfect predictor (AUC = 1.0). Some real churners will still be missed and some loyal customers will still be flagged incorrectly. As noted in Section 2, this is consistent with the churn-prediction literature: behaviour data alone rarely allows AUC values above 0.75 on this kind of dataset, because human decisions to switch provider depend on factors that are not in the file (competitor offers, network quality, family or job circumstances). The strength of the model is therefore not absolute accuracy but the useful ranking it produces, which the retention team can use to prioritise its limited budget.

8. Discussion

8.1 Interpretation of Findings

The comparison of the four model families on a common validation set produced a clear and interpretable ordering. The two gradient boosting systems, XGBoost (AUC = 0.6772) and LightGBM (AUC = 0.6765), led the field, followed by Random Forest (0.6586) and then Logistic Regression (0.6248). The gap between the boosting models and the linear baseline is the most informative part of this result: it shows that the predictive signal in the Cell2Cell data is genuinely multi-factorial and spread across weak, interacting features, which is exactly the situation in which gradient boosting outperforms a linear decision boundary. At the same time, the gap between the boosting models and the much simpler Random Forest is modest, and the two boosting models are statistically indistinguishable from each other. The practical conclusion is that the choice of model family matters, but only up to a point: once a capable tree ensemble is used, further gains come less from the algorithm than from how class imbalance and the decision threshold are handled.

The drivers the model relies on are coherent with domain knowledge and with the wider literature. Equipment age is the single strongest predictor: an ageing handset is a natural trigger for a customer to compare competing offers. Tenure works in the opposite direction, with long-standing customers being markedly more loyal. Recent declines in usage act as an early-warning signal of disengagement that often precedes the cancellation itself, and recurring charge captures price sensitivity among the more expensive customers. That these business-meaningful variables, rather than opaque statistical artefacts, occupy the top of both importance rankings is what makes the model trustworthy enough to inform a retention strategy.

8.2 Answers to Research Questions

The central research question asked to what extent a supervised model trained only on the behavioural, contractual and demographic attributes in the Cell2Cell dataset can predict churn well enough to support a real retention strategy. On the evidence of this project the answer is a qualified yes. The final model does not achieve high absolute accuracy, its ROC-AUC of about 0.67 is moderate, but it produces a stable ranking of customers by churn risk. The three sub-questions are answered in turn below.

RQ1 — Which family of classifiers produces the best ranking of customers by churn risk, and how large is the gap between them? As shown by the baseline comparison in Section 7.1, gradient boosting is the strongest family on this dataset. XGBoost (AUC = 0.6772) and LightGBM (AUC = 0.6765) clearly outperformed Logistic Regression (0.6248) and modestly outperformed Random Forest (0.6586). The gap between the two boosting models is within the noise of a single split, so they are effectively tied, while the gap over the linear baseline is the most substantial and reflects the multi-factorial nature of the churn signal.

RQ2 — Which combination of class weighting, threshold tuning and synthetic oversampling yields the most useful recall–precision trade-off? As shown in Sections 7.2 and 7.3, the most useful trade-off is achieved by class weighting combined with a tuned decision threshold of 0.49. SMOTE did not improve the validation ranking (AUC fell slightly from 0.6772 to 0.6740) and

sharply reduced recall at the default threshold, so synthetic oversampling adds no value on this moderately imbalanced dataset.

RQ3 — Which attributes drive the predictions, and are they consistent across interpretation methods? As shown in Section 7.5, equipment age (CurrentEquipmentDays) and tenure (MonthsInService) dominate both built-in feature importance and permutation importance, with retention contact, recent usage change and recurring charge also prominent. The agreement between the two methods, and with the exploratory analysis of Section 6, indicates that these are genuine and method-stable drivers rather than artefacts.

8.3 Practical Implications

The practical use of the model follows directly from Section 7.6: the ranked list of churn candidates allows the limited retention budget to be prioritised on the customers most likely to leave. The drivers identified in Section 7.5, equipment age, tenure, retention contact and recurring charge, are business-meaningful and stable across interpretation methods, which makes them trustworthy enough to inform retention actions such as device upgrades for customers with ageing handsets or targeted offers for high-charge customers. The ranked list of flagged customers is the deliverable the retention team can act upon.

9. Limitations

Several limitations have to be mentioned. The age of the dataset, already noted in Section 5.1, has direct consequences for the predictive value of the model: customer behaviour, tariff structures and the competitive landscape have changed considerably since the early 2000s, so the model cannot capture phenomena such as modern data plans, streaming usage or bundled offers. The results should therefore be read as a methodological illustration rather than as up-to-date predictions for any present-day operator.

Second, the dataset describes each customer as a single static snapshot. Real customer behaviour evolves over time, and a series of monthly observations per customer would almost certainly allow stronger predictions through time-series features or survival models. With only one row per customer, such effects cannot be modelled here.

Third, the absolute predictive performance of the final model is moderate, with a validation ROC-AUC of about 0.67. As discussed in Section 7.6, churn also depends on external factors that are absent from the dataset, and no model trained only on this file can fully capture those drivers.

Finally, ServiceArea was excluded from the final model because of its very high cardinality. This means that potentially informative geographical patterns were not considered. Future work could aggregate individual area codes into broader regions and evaluate whether geographical information improves churn prediction performance.

10. Conclusion

This project followed the full machine-learning workflow set out in Sections 3 and 4, from data inspection and cleaning through to the interpretation of a final tuned model. Among the four model families compared in Section 7.1, the two gradient boosting libraries (XGBoost and LightGBM) clearly outperformed Logistic Regression and modestly outperformed Random Forest. A business-aware decision threshold of 0.49 was chosen in Section 7.2, SMOTE was tested in Section 7.3 but did not improve the validation ranking, and the final LightGBM model was tuned and interpreted in Sections 7.4 and 7.5.

In direct response to the research questions: RQ1 is answered by the finding that gradient boosting is the strongest classifier family on this dataset, with XGBoost and LightGBM effectively tied and well ahead of the linear baseline. RQ2 is answered by the finding that class weighting combined with a tuned threshold of 0.49 gives the most useful recall–precision trade-off, while SMOTE adds no value. RQ3 is answered by the finding that equipment age and tenure, followed by retention contact, recent usage change and recurring charge, are the dominant drivers and remain stable across both built-in and permutation importance.

Two lessons stand out. First, tree-based gradient boosting models outperformed linear models on this dataset, which is consistent with the wider literature on tabular data and reflects the multi-factorial nature of churn: many weak signals combine into a useful prediction. Second, interpretability is not a luxury but a requirement: a churn score is only acted upon if the retention team understands why a customer was flagged, and built-in importance together with permutation importance provides exactly that bridge between the model and the business.

The final model achieves a validation ROC-AUC of about 0.67 and flags roughly 48% of the holdout customers as churn candidates. This level of performance is moderate but useful in practice, because the value of the model lies in the ranking it produces, not in absolute correctness. With this ranking the retention budget can be spent on the customers who are most likely to leave, which is the original business goal of the project.

11. References

1. Bergstra, J. and Bengio, Y. (2012) ‘Random Search for Hyper-Parameter Optimization’, Journal of Machine Learning Research, 13, pp. 281–305.

2. Bischl, B. et al. (2023) ‘Hyperparameter Optimization: Foundations, Algorithms, Best Practices, and Open Challenges’, WIREs Data Mining and Knowledge Discovery, 13(2), e1484.

3. Breiman, L. (2001) ‘Random Forests’, Machine Learning, 45(1), pp. 5–32.

4. Chawla, N. V., Bowyer, K. W., Hall, L. O. and Kegelmeyer, W. P. (2002) ‘SMOTE: Synthetic Minority Over-sampling Technique’, Journal of Artificial Intelligence Research, 16, pp. 321–357.

5. Chen, T. and Guestrin, C. (2016) ‘XGBoost: A Scalable Tree Boosting System’, in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. New York: ACM, pp. 785–794.

6. Fawcett, T. (2006) ‘An introduction to ROC analysis’, Pattern Recognition Letters, 27(8), pp. 861–874.

7. Harris, C. R. et al. (2020) ‘Array programming with NumPy’, Nature, 585, pp. 357–362.

8. Hastie, T., Tibshirani, R. and Friedman, J. (2009) The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd edn. New York: Springer.

9. Hunter, J. D. (2007) ‘Matplotlib: A 2D Graphics Environment’, Computing in Science & Engineering, 9(3), pp. 90–95.

10. Kohavi, R. (1995) ‘A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection’, in Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI). San Francisco: Morgan Kaufmann, pp. 1137–1143.

11. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y. (2017) ‘LightGBM: A Highly Efficient Gradient Boosting Decision Tree’, in Advances in Neural Information Processing Systems 30 (NIPS 2017). Long Beach: Curran Associates.

12. Lemaître, G., Nogueira, F. and Aridas, C. K. (2017) ‘Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning’, Journal of Machine Learning Research, 18(17), pp. 1–5.

13. LightGBM Developers (2024) LightGBM Documentation. Available at: https://lightgbm.readthedocs.io/ (Accessed: 14 May 2026).

14. Little, R. J. A. and Rubin, D. B. (2019) Statistical Analysis with Missing Data. 3rd edn. Hoboken: Wiley.

15. López, V., Fernández, A., García, S., Palade, V. and Herrera, F. (2013) ‘An insight into classification with imbalanced data: Empirical results and current trends on using data intrinsic characteristics’, Information Sciences, 250, pp. 113–141.

16. Lundberg, S. M. and Lee, S.-I. (2017) ‘A Unified Approach to Interpreting Model Predictions’, in Advances in Neural Information Processing Systems 30 (NIPS 2017). Long Beach: Curran Associates.

17. Manning, C. D., Raghavan, P. and Schütze, H. (2008) Introduction to Information Retrieval. Cambridge: Cambridge University Press.

18. McKinney, W. (2010) ‘Data Structures for Statistical Computing in Python’, in Proceedings of the 9th Python in Science Conference (SciPy), pp. 56–61.

19. Neslin, S. A., Gupta, S., Kamakura, W., Lu, J. and Mason, C. H. (2006) ‘Defection Detection: Measuring and Understanding the Predictive Accuracy of Customer Churn Models’, Journal of Marketing Research, 43(2), pp. 204–211.

20. Pedregosa, F. et al. (2011) ‘Scikit-learn: Machine Learning in Python’, Journal of Machine Learning Research, 12, pp. 2825–2830.

21. Scikit-learn Developers (2024) Scikit-learn User Guide: Pipelines, Preprocessing, Model Selection and Metrics. Available at: https://scikit-learn.org/stable/user_guide.html (Accessed: 14 May 2026).

22. Verbeke, W., Dejaeger, K., Martens, D., Hur, J. and Baesens, B. (2012) ‘New insights into churn prediction in the telecommunication sector: A profit driven data mining approach’, European Journal of Operational Research, 218(1), pp. 211–229.

23. Waskom, M. L. (2021) ‘seaborn: statistical data visualization’, Journal of Open Source Software, 6(60), 3021.

24.XGBoost Developers (2024) XGBoost Documentation. Available at: https://xgboost.readthedocs.io/ (Accessed: 14 May 2026).

25. Kaggle (2018) Telecom churn dataset (Cell2Cell), uploaded by user jpacse. Available at: https://www.kaggle.com/datasets/jpacse/datasets-for-churn-telecom

26 Breiman, L., Friedman, J. H., Olshen, R. A. and Stone, C. J. (1984) → goes right **before** your existing "Breiman, L. (2001)" entry (1984 comes before 2001).

27. Shannon, C. E. (1948) → goes between "Scikit-learn Developers (2024)" and "Verbeke..." — alphabetically under S, after Scikit-learn.